HANDWRITTEN DIGIT RECOGNITION

A modern comparison of two convolutional networks

DLBAIPCV01 - Project: Computer Vision

30 October 2026

Author: Mubeen Ahmed Soomro

Abstract

This project compares Chollet's Keras CNN with a compact modern version of Le Cun et al.'s 1989 architecture on MNIST. Both used the same preprocessing, Adam optimizer, batch size of 128, 15 epochs, and 10% validation split. On the 10,000-image test set, Chollet's model reached 99.06% accuracy (loss 0.02757); the compact model reached 98.32% (loss 0.05131). It used 3,246 parameters, 90.7% fewer than Chollet's 34,826, and its fit time was 58.5 seconds compared with 166.4 seconds in this CPU run. Chollet's model was more accurate, while the compact model used fewer parameters and trained faster. The Le Cun-inspired model is an approximation that uses modern Keras layers and dropout in place of the historical sparse connections. Each model was trained once, so these results support a descriptive comparison but not a claim of statistical significance or expected performance across runs.

Graphical abstract

MNIST INPUT	TWO KERAS CNNs	COMMON PROTOCOL	HELD-OUT RESULT
28 x 28 grayscale 60,000 train / 10,000 test	Chollet baseline LeCun-inspired compact model	Adam | 128 batch 15 epochs | 10% validation	99.06% vs 98.32% 34,826 vs 3,246 parameters

The higher-capacity model led on test accuracy; the compact model used 90.7% fewer parameters and trained faster in this run.

1. Introduction and related work

Handwritten digit recognition is a compact but meaningful computer-vision problem: a model must map variable human-written shapes to one of ten labels. Convolutional networks use local receptive fields and shared weights to reuse feature detectors across image positions. Le Cun et al. (1989, pp. 399-400) describe these constraints as a way to encode prior knowledge about shape and reduce the number of independent parameters. Their architecture alternates feature extraction with local averaging and subsampling.

The original study addressed U.S. postal ZIP-code digits, not MNIST. Its data included handwritten and printed numerals; the reported test error therefore does not provide a like-for-like score against this project. The paper reports its test-set outcomes on its own postal data (Le Cun et al., 1989, pp. 397, 402). This project instead uses MNIST to compare two modern Keras implementations under one shared evaluation protocol.

Chollet's Keras example is a simple two-convolution MNIST classifier. It scales pixel intensities to [0, 1], uses 32 then 64 3 x 3 convolution filters with max pooling, applies 50% dropout, and ends with a ten-unit softmax classifier (Chollet, 2021). The Keras dataset documentation identifies 60,000 28 x 28 training images and 10,000 test images across labels 0-9 (Keras, n.d.).

Project question

How does a compact, LeCun-inspired Keras network compare with Chollet's reference CNN on MNIST test accuracy, cross-entropy loss, parameter count, training time, and class-level errors when the data split and training schedule are held constant?

Contribution and scope

The project makes the adaptation explicit, reports both model settings, measures rather than assumes the results, and includes reproducible code. It does not claim to recreate the original hardware, optimizer, data, or exact sparse H2-to-H3 connection scheme. The two architectures are compared on MNIST using the same modern training setup.

2. Method

Dataset and preparation

The experiment uses Keras' fixed MNIST split: 60,000 training images and 10,000 test images, each 28 x 28 grayscale pixels. The training images are converted to float32 and divided by 255. A trailing channel dimension changes the tensors to 28 x 28 x 1. The first 90% of the training array is used for fitting and the remaining 10% for validation, matching the Keras `validation_split` approach. The test split is not used during fitting or model selection.

Training protocol

Both models use categorical cross-entropy, Adam, batch size 128, 15 epochs, shuffle enabled, and the same 10% validation split. A fixed seed (2026) initializes the random generators; deterministic TensorFlow operations are enabled where supported. Training time is the wall-clock duration of `model.fit`, excluding the final test evaluation. The experiment ran with TensorFlow 2.18.0 and Python 3.12.14 on a Windows Intel CPU.

Pipeline



Figure 1. Shared data and evaluation flow for both model runs.

Model evaluation

The report includes test loss and accuracy, total trainable parameters, training time, learning curves, 10 x 10 confusion matrices, and per-class recall. Test-set comparisons are descriptive for the recorded seed and machine.

3. Architectures and implementation choices

The LeCun-inspired network follows the original sequence of shared-weight feature maps and local subsampling while keeping the model small. H1 has four 5 x 5 maps; average pooling forms H2. H3 has twelve 5 x 5 tanh maps; a 25% dropout layer is a modern regularization proxy for the original sparse H2-to-H3 connections, followed by average pooling as H4. The output uses a ten-class softmax. Le Cun et al. describe four H1 maps, four H2 subsampling planes, twelve H3 maps, and twelve H4 planes (1989, pp. 400-402).

Table 1. Source architecture and the evaluated Keras implementations

Setting	Le Cun et al. (1989) original	Chollet (2021) reference	LeCun-inspired implementation
Dataset	USPS digits; historical postal data	MNIST	MNIST
Input	28 x 28 plane; normalized active digit	28 x 28 x 1	28 x 28 x 1
Feature maps	H1: 4 maps, 5 x 5; H3: 12 maps, 5 x 5	32 then 64 filters, 3 x 3	4 then 12 filters, 5 x 5
Subsampling	H2 and H4 local averaging, 2:1	Two 2 x 2 max pools	Two 2 x 2 average pools
Inter-map links	Sparse fixed H2-to-H3 table	Fully connected convolutional channels	Dense channel convolution + 25% dropout proxy
Classifier	H4 fully connected to 10 outputs	Dense 10, softmax	Dense 10, softmax
Optimizer / schedule	Second-order backprop; 30 passes reported	Adam; 128 batch; 15 epochs	Adam; 128 batch; 15 epochs
Trainable parameters	2,578 reported for original model	34,826 measured	3,246 measured

The LeCun-inspired model has 3,246 trainable parameters: 104 in H1, 1,212 in H3, and 1,930 in the classifier. Chollet's model has 34,826: 320 in the first convolution, 18,496 in the second convolution, and 16,010 in the classifier. Pooling, flattening, and dropout add no trainable weights.

4. Results

Each model was trained once with the same seed and protocol. Table 2 reports values from the final test evaluation; the epoch-level histories are saved in the project results file.

Model	Parameters	Fit time (s)	Test loss	Test accuracy	Correct / 10,000
Chollet (2021)	34,826	166.4	0.02757	99.06%	9,906
LeCun et al. (1989), modern approximation	3,246	58.5	0.05131	98.32%	9,832

The reference model was higher by 0.74 percentage points on this test run and had lower cross-entropy loss. The compact model used 9.3% as many parameters (a 90.7% reduction) and completed fitting in 58.5 seconds versus 166.4 seconds on this machine. Fit times depend on the processor, software build, and system load; they are not general hardware benchmarks.

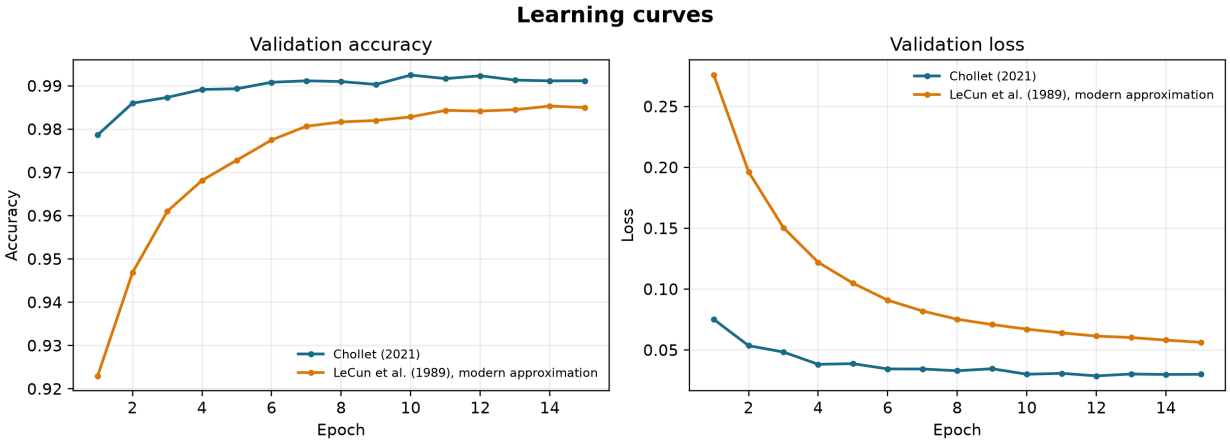


Figure 2. Validation accuracy and cross-entropy loss across the 15 training epochs.

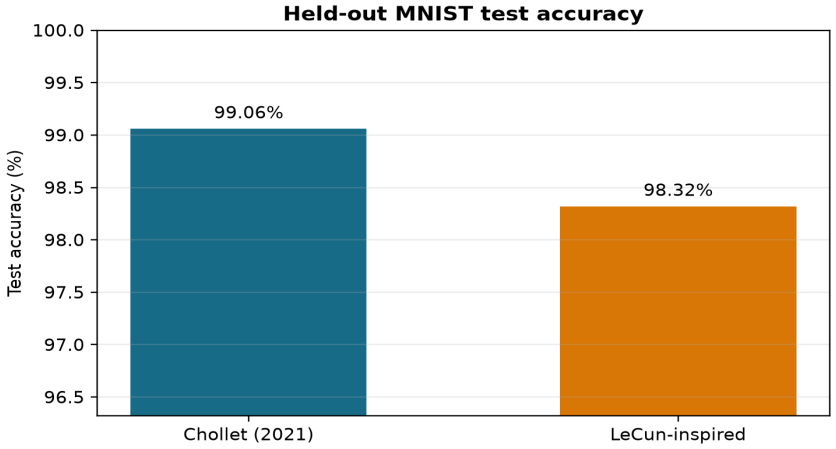


Figure 3. Held-out MNIST test accuracy. The axis is truncated to make the small difference visible.

5. Error analysis

The confusion matrices show that errors are uncommon for both classifiers. The largest individual off-diagonal count for Chollet's model was true 6 predicted as 0 (8 examples). For the compact model, the largest was true 7 predicted as 2 (13 examples); true 9 predicted as 7 and true 9 predicted as 4 each occurred 8 times. These counts describe this test run and should not be interpreted as class-wide causes.

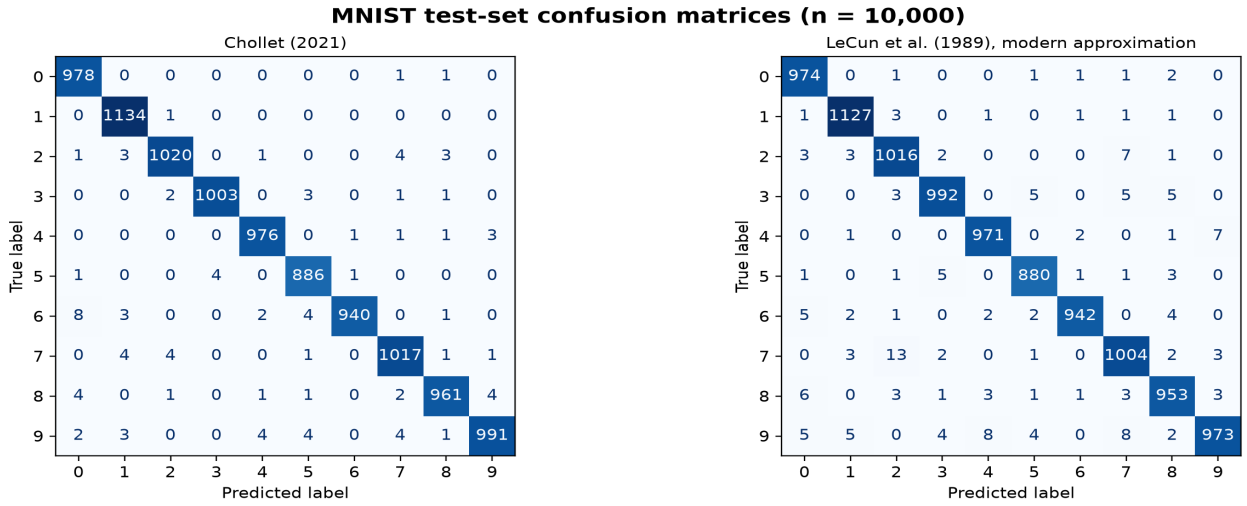


Figure 4. Confusion matrices on the 10,000-image test split. Rows are true labels and columns are predictions.

Table 3. Per-class recall on the test split

Digit	Chollet (2021)	LeCun-inspired
0	99.80%	99.39%
1	99.91%	99.30%
2	98.84%	98.45%
3	99.31%	98.22%
4	99.39%	98.88%
5	99.33%	98.65%
6	98.12%	98.33%
7	98.93%	97.67%
8	98.67%	97.84%
9	98.22%	96.43%

The lowest recall for Chollet's model was digit 6 (98.12%), followed by digit 9 (98.22%). For the compact model, digit 9 was lowest (96.43%), followed by digit 7 (97.67%). The model-level difference is consistent with the total accuracy gap, while most individual classes remain above 96%.

6. Discussion

Accuracy and model size

Chollet's network scored 99.06% accuracy on the test set, compared with 98.32% for the LeCun-inspired model. The smaller model used 3,246 parameters, about one-tenth of Chollet's 34,826, and had a shorter measured fit time. If test accuracy is the priority on MNIST, Chollet's model performed better in this run. If parameter count or training time matters more, the compact model may be a reasonable option. Memory use, inference latency, and the cost of different errors were not measured.

What the historical comparison can and cannot say

The original paper reports 2,578 independent parameters and evaluates on postal-code digits, whereas the modern approximation is trained on MNIST. Differences between its historical result and the present numbers would mix dataset, preprocessing, implementation, hardware, and optimizer effects. Therefore the quantitative comparison is between the two Keras models on MNIST; the 1989 network is the architectural reference rather than a third MNIST benchmark (Le Cun et al., 1989, pp. 397, 400-402).

Limitations

Only one run was performed for each model. The results show what happened with seed 2026 on this machine, not how much scores vary across runs. A few repeated runs would be needed to estimate that variation. Dropout is a modern stand-in for the original sparse links between H2 and H3; it does not reproduce the fixed connection table. The model keeps the paper's feature-map scale and average subsampling but uses dense channel convolution and a modern optimizer. Both models use Adam and 15 epochs rather than the original training procedure. Fit times depend on the CPU and TensorFlow build. No hyperparameter search was done, so the results also depend on the settings selected here.

Reproducibility

The project records its seed, package versions, model definitions, test metrics, confusion matrices, per-epoch histories, and layer parameters. Running `project_cv.py` downloads MNIST via Keras on first use and regenerates the figures and JSON results. Exact training time can vary by system.

7. Project process, resources, and reflection

The work was organized as a sequence of checkable stages. Each stage produced an artifact used in the next, which kept the comparison reproducible and made deviations visible.

Stage	Decision and evidence
Scope and sources	Defined a shared MNIST question; checked the historical paper and the published Keras baseline.
Implementation	Built both models and applied the same split, preprocessing, seed, optimizer, batch size, and epoch count.
Measurement	Recorded held-out loss, accuracy, parameter counts, fit time, learning curves, and class-level errors.
Evaluation and reporting	Compared accuracy against model size; documented the architecture proxy, limits, and reproducibility steps.

Resources and constraints

Resources used

The work used the public Keras MNIST dataset, Python 3.12, TensorFlow/Keras 2.18, and a Windows Intel CPU. No paid data or cloud compute was used in the recorded run; no monetary budget or labor-hour log was maintained. The shared code, environment requirements, saved metrics, and figures support review and rerunning.

Risks and response

The main validity risk was treating a modern implementation as an exact historical reproduction. The report limits that claim and identifies dropout as a proxy for sparse links. A second risk is run-to-run and machine variation; one seeded run is reported descriptively, while repeated seeds and a direct sparse-mask implementation are proposed as follow-up work.

The project achieved its comparison objective and delivered runnable code, saved outputs, and a documented interpretation. The main improvement for a subsequent iteration is to plan repeated runs and a closer historical implementation before drawing broader conclusions.

8. Conclusion

Chollet's CNN reached 99.06% test accuracy and 0.02757 loss. The LeCun-inspired Keras model reached 98.32% and 0.05131, while using 90.7% fewer parameters and taking less time to fit in this run. Chollet's model performed better when accuracy was the priority; the compact model offered a smaller network with a modest loss in accuracy. This result comes from one seeded run, and the historical network was approximated rather than reproduced exactly.

The next useful extension would be to repeat the experiment across several seeds and report the mean and standard deviation for test accuracy and fit time. A second extension would implement the original H2-to-H3 sparse mask directly, separating the effect of that historical constraint from dropout regularization.

References

Chollet, F. (2021). *Simple MNIST convnet*. Keras. https://keras.io/examples/vision/mnist_convnet/

Keras. (n.d.). *MNIST digits classification dataset*. Keras API. https://keras.io/api/datasets/mnist/

Le Cun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., & Jackel, L. D. (1989). Handwritten digit recognition with a back-propagation network. In *Advances in Neural Information Processing Systems 2* (pp. 396-404). Morgan Kaufmann. https://proceedings.neurips.cc/paper_files/paper/1989/file/53c3bce66e43be4f209556518c2fcb54-Paper.pdf

Appendix A. Python source code

Complete experiment module. The same editable source is supplied as **project_cv.py**. To reproduce the result, run it in the environment listed in README.md.

```
"""MNIST CNN comparison for DLBAIPCV01 Project: Computer Vision.

Models: Chollet's published Simple MNIST ConvNet and a modern Keras approximation
of the LeCun et al. (1989) convolutional architecture.
"""
from __future__ import annotations

import json
import os
import platform
import time
from pathlib import Path

os.environ.setdefault("TF_CPP_MIN_LOG_LEVEL", "2")
os.environ.setdefault("TF_ENABLE_ONEDNN_OPTS", "1")

import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

ROOT = Path(__file__).resolve().parent
RESULTS_DIR = ROOT / "results"
FIGURES_DIR = ROOT / "figures"
SEED = 2026
EPOCHS = 15
BATCH_SIZE = 128


def load_mnist():
    """Load the canonical 60k/10k MNIST split and apply shared preprocessing."""
    (x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
    x_train = (x_train.astype("float32") / 255.0)[..., np.newaxis]
    x_test = (x_test.astype("float32") / 255.0)[..., np.newaxis]
    y_train_oh = keras.utils.to_categorical(y_train, 10)
    y_test_oh = keras.utils.to_categorical(y_test, 10)
    return (x_train, y_train, y_train_oh), (x_test, y_test, y_test_oh)


def build_chollet_model():
    """Chollet (2021) Simple MNIST ConvNet, reproduced from the published example."""
    model = keras.Sequential(
        [
            keras.Input(shape=(28, 28, 1), name="digit_image"),
            layers.Conv2D(32, kernel_size=(3, 3), activation="relu", name="conv_1"),
            layers.MaxPooling2D(pool_size=(2, 2), name="max_pool_1"),
            layers.Conv2D(64, kernel_size=(3, 3), activation="relu", name="conv_2"),
            layers.MaxPooling2D(pool_size=(2, 2), name="max_pool_2"),
            layers.Flatten(name="flatten"),
            layers.Dropout(0.5, seed=SEED, name="dropout"),
            layers.Dense(10, activation="softmax", name="class_probabilities"),
        ],
        name="chollet_simple_mnist_convnet",
    )
    model.compile(
        loss="categorical_crossentropy",
        optimizer=keras.optimizers.Adam(),
        metrics=["accuracy"],
    )
    return model


def build_lecun_approximation():
    """LeCun-inspired feature extraction and subsampling, implemented in Keras.

    H1: four 5x5 shared-weight feature maps. H2: 2x2 local averaging/subsampling.
    H3: twelve 5x5 feature maps. Dropout is an explicit modern regularization
    proxy for the historical H2-to-H3 sparse connectivity; it is not an exact
    structural reproduction of the original connection table. H4: subsampling.
    """
    model = keras.Sequential(
        [
            keras.Input(shape=(28, 28, 1), name="digit_image"),
            layers.Conv2D(4, kernel_size=(5, 5), activation="tanh", name="H1_features"),
            layers.AveragePooling2D(pool_size=(2, 2), name="H2_subsampling"),
            layers.Conv2D(12, kernel_size=(5, 5), activation="tanh", name="H3_features"),
            layers.Dropout(0.25, seed=SEED, name="H2_H3_connectivity_proxy"),
            layers.AveragePooling2D(pool_size=(2, 2), name="H4_subsampling"),
            layers.Flatten(name="flatten"),
            layers.Dense(10, activation="softmax", name="class_probabilities"),
        ],
        name="lecun_1989_modern_approximation",
    )
    model.compile(
        loss="categorical_crossentropy",
        optimizer=keras.optimizers.Adam(),
        metrics=["accuracy"],
    )
```

```
    return model


def _run_one(name, model, x_train, y_train_oh, x_test, y_test_oh, y_test):
    start = time.perf_counter()
    history = model.fit(
        x_train,
        y_train_oh,
        batch_size=BATCH_SIZE,
        epochs=EPOCHS,
        validation_split=0.1,
        shuffle=True,
        verbose=2,
    )
    training_seconds = time.perf_counter() - start
    test_loss, test_accuracy = model.evaluate(x_test, y_test_oh, verbose=0)
    probabilities = model.predict(x_test, batch_size=512, verbose=0)
    predictions = probabilities.argmax(axis=1)
    cm = tf.math.confusion_matrix(y_test, predictions, num_classes=10).numpy()
    return {
        "name": name,
        "parameters": int(model.count_params()),
        "training_seconds": float(training_seconds),
        "test_loss": float(test_loss),
        "test_accuracy": float(test_accuracy),
        "history": {k: [float(v) for v in vals] for k, vals in history.history.items()},
        "confusion_matrix": cm.tolist(),
        "predictions": predictions.tolist(),
    }


def run_experiment():
    """Train both models once and save the machine-readable measurements/plots."""
    RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    FIGURES_DIR.mkdir(parents=True, exist_ok=True)
    keras.utils.set_random_seed(SEED)
    try:
        tf.config.experimental.enable_op_determinism()
    except (AttributeError, RuntimeError):
        pass

    (x_train, y_train, y_train_oh), (x_test, y_test, y_test_oh) = load_mnist()
    print(f"Dataset: train={x_train.shape}, test={x_test.shape}; seed={SEED}")
    print(f"Runtime: TensorFlow {tf.__version__}; Python {platform.python_version()}; {platform.processor() or
    platform.machine()}")

    models = [
        ("Chollet (2021)", build_chollet_model()),
        ("LeCun et al. (1989), modern approximation", build_lecun_approximation()),
    ]
    model_results = []
    for name, model in models:
        print(f"\nTraining {name} ({model.count_params():,} parameters)")
        model_results.append(_run_one(name, model, x_train, y_train_oh, x_test, y_test_oh, y_test))

    payload = {
        "experiment": "MNIST CNN comparison",
        "seed": SEED,
        "epochs": EPOCHS,
        "batch_size": BATCH_SIZE,
        "validation_split": 0.1,
        "training_samples": int(len(x_train)),
        "validation_samples": int(len(x_train) * 0.1),
        "test_samples": int(len(x_test)),
        "tensorflow_version": tf.__version__,
        "python_version": platform.python_version(),
        "platform": platform.platform(),
        "models": model_results,
    }
    result_path = RESULTS_DIR / "experiment_results.json"
    result_path.write_text(json.dumps(payload, indent=2), encoding="utf-8")
    _save_figures(payload)
    _save_model_parameter_table(models)
    print("\nFinal test results")
    for item in model_results:
        print(
            f"{item['name']}: loss={item['test_loss']:.5f}, "
            f"accuracy={item['test_accuracy']:.5f}, "
            f"fit={item['training_seconds']:.1f}s, params={item['parameters']:,}"
        )
    print(f"Saved results to {result_path}")
    return payload


def _save_figures(payload):
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    from sklearn.metrics import ConfusionMatrixDisplay

    fig, axes = plt.subplots(1, 2, figsize=(12, 4.8), constrained_layout=True)
    for ax, item in zip(axes, payload["models"]):
        ConfusionMatrixDisplay(
            confusion_matrix=np.asarray(item["confusion_matrix"]),
            display_labels=np.arange(10),
```

```
        ).plot(ax=ax, colorbar=False, values_format="d", cmap="Blues")
        ax.set_title(item["name"], fontsize=10)
    fig.suptitle("MNIST test-set confusion matrices (n = 10,000)", fontsize=14, weight="bold")
    fig.savefig(FIGURES_DIR / "confusion_matrices.png", dpi=220, bbox_inches="tight")
    plt.close(fig)

    fig, axes = plt.subplots(1, 2, figsize=(11, 4), constrained_layout=True)
    colors = ["#176B87", "#D97706"]
    for ax, metric, label in zip(axes, ["accuracy", "loss"], ["Accuracy", "Loss"]):
        for color, item in zip(colors, payload["models"]):
            key = f"val_{metric}"
            ax.plot(range(1, len(item["history"][metric]) + 1), item["history"][key],
                    marker="o", markersize=3, linewidth=1.8, color=color, label=item["name"])
        ax.set_xlabel("Epoch")
        ax.set_ylabel(label)
        ax.grid(alpha=0.25)
        ax.legend(frameon=False, fontsize=8)
        ax.set_title(f"Validation {label.lower()}")
    fig.suptitle("Learning curves", fontsize=14, weight="bold")
    fig.savefig(FIGURES_DIR / "learning_curves.png", dpi=220, bbox_inches="tight")
    plt.close(fig)

    fig, ax = plt.subplots(figsize=(6.5, 3.8), constrained_layout=True)
    names = ["Chollet (2021)", "LeCun-inspired"]
    acc = [m["test_accuracy"] * 100 for m in payload["models"]]
    bars = ax.bar(names, acc, color=colors, width=0.55)
    ax.set_ylim(max(90, min(acc) - 2), 100)
    ax.set_ylabel("Test accuracy (%)")
    ax.set_title("Held-out MNIST test accuracy", weight="bold")
    ax.grid(axis="y", alpha=0.25)
    for bar, value in zip(bars, acc):
        ax.text(bar.get_x() + bar.get_width() / 2, value + 0.1, f"{value:.2f}%", ha="center", fontsize=10)
    fig.savefig(FIGURES_DIR / "accuracy_comparison.png", dpi=220, bbox_inches="tight")
    plt.close(fig)


def _save_model_parameter_table(models):
    (x_train, _, _), (x_test, _, _) = load_mnist()
    rows = []
    for name, model in models:
        for layer in model.layers:
            rows.append({"model": name, "layer": layer.name, "output_shape": str(layer.output.shape), "parameters":
    int(layer.count_params())})
    out = RESULTS_DIR / "layer_parameters.csv"
    import csv
    with out.open("w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=["model", "layer", "output_shape", "parameters"])
        writer.writeheader()
        writer.writerows(rows)


if __name__ == "__main__":
    run_experiment()
```